
Press Esc to exit full screen
System Verilog for Design and Verification
Amr Mohamed Abbas
QuestaSim QA Manager
05:47
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential
SIEMENS
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)
Abbas, Amr Mohamed (DI ...

Course Outline

SV for Design	SV for Verification
Data Types	Classes
Arrays & Structures	Interfaces
SV Scheduler	Randomization & Constraints
Program Control	SVA
Hierarchy	
Dynamic Processes	

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

SystemVerilog

SystemVerilog is a combination of both Hardware Description Language (HDL) and Hardware Verification Language (HVL) and combined termed as HDVL. Means it describe the structure and behavior of electronic circuits as well as it verifies the electronic circuits written in a Hardware Description Language.
SystemVerilog acts as a superset of Verilog with a lot extensions to Verilog language in 2005 and became IEEE standard 1800 and again updated in 2012 as IEEE 1800-2012 standard. SystemVerilog is based on class level testbench which is more dynamic in nature.

Verilog	System Verilog
Verilog is a Hardware Description Language (HDL).	SystemVerilog is a combination of both Hardware Description Language (HDL) and Hardware Verification Language (HVL).
Verilog language is used to structure and model electronic systems.	SystemVerilog language is used to model, design, simulate, test and implement electronic system.
It supports structured paradigm.	It supports structured and object oriented paradigm.
Verilog is based on module level testbench.	SystemVerilog is based on class level testbench.
It is standardized as IEEE 1364.	It is standardized as IEEE 1800-2012.
Verilog is influenced by C language.	SystemVerilog is based on Verilog, VHDL and c++ programming language.
It has file extension .v or .vh	It has file extension .sv or .svh
It supports Wire and Reg datatype.	It supports various datatypes like enum, union, struct, string, class.
It is based on hierarchy of modules.	It is based on classes.
It was began in 1983 as proprietary language for hardware modelling.	It was originally intended as an extension to Verilog in the year 2005.

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential
SIEMENS

Data Types

In this section



- Data types
- Static & Dynamic Variables
- Enumeration



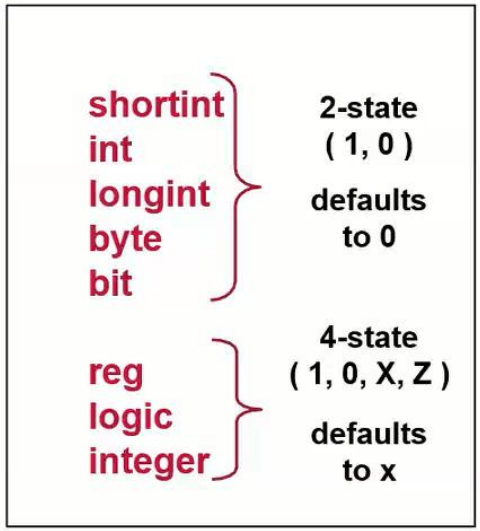
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

SystemVerilog Data Types

time	64-bit integer, defaults to seconds
real	from Verilog, like C double, 64-bits
shortreal	from C float, 32-bits
string	variable size array of characters
void	non-existent data, used for functions



shortint	16-bit integer
int	32-bit integer
longint	64-bit integer
byte	8-bit integer (ASCII character)
integer	32-bit from Verilog (sized: 0, 1, X, Z)

default to signed

`int unsigned usig;` *// make unsigned*

bit	0 or 1
reg	from Verilog (unsized: 0, 1, X, Z)
logic	like reg

default to unsigned

`reg signed ssig;` *// make signed*

Press Ctrl+Shift+M to unmute your microphone.

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

User Defined Types

- SystemVerilog supports a new keyword: **typedef**

Syntax:
```
typedef <base_data_type> <type_identifier>
```

```
typedef int typ ;          // typ becomes a new type
typ a = 1, b = 10;         // these are 2 new variables of type 'typ'
```



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Enumeration

Syntax:

```
enum [enum_base_type] { enum_name_declaration
{,enum_name_declaration} }

enum_base_type:  default is int
```

- Enumeration is a useful way of defining abstract variables.

NOTE:

Default assigned values start at zero

```
          0       1          2
enum  {red, green, yellow} lite;
```

- Define an enumeration with " enum "

```
enum  {red, green, yellow} traf_lite1, traf_lite2;
```

- Values can be cast to integer types and auto-incremented

```
enum  { a=5, b, c} vars;    // b=6, c=7
```

- A sized constant can be used to set size of the type

```
enum bit[3:0] {bronze=4'h3, silver, gold} medal;
      // All medal members are (must be) 4-bits
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Enumerated Types

- **Enumerated Type**
  - Define a new type

```
typedef enum {NO, YES} boolean;     // boolean is NOT a SystemVerilog type
boolean myvar;                      // but it just became one ☺
```

"myvar" will be checked for valid values in all assignments

```
module enum_types;
typedef enum {red, green, blue, yellow, white, black} colors ;
colors my_colors;

initial begin
  my_colors = green;
  my_colors = 1;            // is this valid ?
  my_colors = colors'(2);   // casting
  case (my_colors)
    red:   $display ("my_color is red");
    blue:  $display ("my_color is blue");
    default:  $display ("my_color is not blue or red");
  endcase
end
endmodule
```

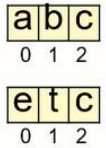

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

String Data Type

- String data type is a variable size array of characters, indexed from 0 to N-1 (N is array length)
  - Every element of the array is also a string

```
string st1, st2;
st1 = "";                    // null string
st2 = "abc";                 // assigned from a string literal
st1 = { st1, st2 };          // concatenation, st1 becomes "abc"
st2[0:1] = "et";             // st2 becomes "etc"
```



- Supports relational operators (`==, !=, <, <=, >, >=`), concatenation (`{ }`) and replication (`{n{ }}`)
  - e.g. (a<b) is true because a precedes b alphabetically
- By means of the . operator, strings support special methods:
  - `len()`, `putc()`, `getc()`, `toupper()`, `tolower()`, `compare()`, `icompare()`, `substr()`, `atoi()`, `atohex()`, `atooct()`, `atobin()`, `atoreal()`, `itoa()`, `hextoa()`, `octtoa()`, `bintoa()` **and** `realtoa()`

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Parameterized types

- SystemVerilog extends Verilog parameters to support types.
  This example shows a fifo, whose width, depth AND TYPE are parameterized:

```
module fifo  #( parameter depth = 16,          // default width/depth
                parameter width = 8,           // default width/depth
                parameter type ft = bit )      // fifo is type bit (2-state) by default
              ( input  ft [width-1:0] datin, input bit r_w, clk,
                output ft [width-1:0] datout );

ft [width-1:0] mem [depth-1:0];                // declare fifo array & type (bit by default)
. . .
endmodule

module use_fifo;
  logic [31:0] i, o;
  bit clk, r_w;

  fifo   #( .depth(64), .width(32),            // override both parameter default values
                .ft(logic) )                   // override parameter ft to be type logic (4-state)
         U1(.datout(o), .datin(i), .clk(clk), .r_w(r_w));

endmodule
```

Tip
An easy way (as here) to switch between 2-state and 4-state operation of your design.

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Arrays & Structures

In this section



- Dynamic Arrays
- Associative Arrays
- Queues
- Structure / Union

- Multi-dimensional
- Using Arrays
- Supported data types & operations
- Querying functions

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

Dynamic Arrays

- Dynamic declaration of one-dimensional arrays

Syntax:

```
data_type array_name[] ;
```

Declares a dynamic array *array_name* of type *data_type*

```
data_type array_name[] = new[ array_size ] [(array)] ;
```

Allocates a new array *array_name* of type *data_type* and size *array_size*
Optionally assigns values of *array* to *array_name*
If no value is assigned then element has default value of *data_type*

Declaration

```
bit [3:0] nibble[ ];     // Dynamic array of 4-bit vectors
int data[ ];             // Dynamic array of int
```

Resize

```
initial begin
  nibble = new[100];     // resize to 100-element array
  data = new [256];      // resize to 256-element array
end
```

Declare & size

```
int addr[ ] = new [50]; // Create a 50-element array
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

Dynamic Arrays – Array Assignments

- Assignment of a dynamic array to a dynamic array
  - Creates a new dynamic array the size of the RHS array

```
int dyn_array[] = new[100];
int  an_array[];

initial begin
  an_array = new[dyn_array.size()](dyn_array);   // same as next line
  an_array = dyn_array;                          // same as line above
  an_array[2] = 222;                             // init location  2
  an_array = new[150];                           // resize array - lose contents
  $display("1: an_array[2] = %0d",an_array[2]);
  an_array[3] = 333;                             // init location 3
  an_array = new[200](an_array);                 // resize array - save contents
  $display("2: an_array[3] = %0d",an_array[3]);
end
```

output:
1: an_array[2] = 0
2: an_array[3] = 333

- Assignment of a fixed size array to a dynamic array
  - OK if data type and array size are the same

```
int odd_size[] = new[87];
int fix_sz_array[100];

initial begin
  dyn_array = fix_sz_array;      // is this Ok ?
  odd_size = fix_sz_array;       // is this Ok ?
end
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Arrays – Associative

- Associative arrays (sometimes called indexed arrays)
  - Support situations where data set size is totally unpredictable and elements may be added or removed individually to grow/shrink the array
  - Implemented as a look up table and so require an index.

Syntax:

```
data_type array_id [ index_type ];    // index type is the datatype to use as index
                                      // examples include string, int, class, struct
```

Example:

```
bit i_array[*];                  // associative array of bits (unspecified index)
                                 // unspecified index (*) implies any integral value
bit [7:0] age [string];          // associative array of 8-bit vectors, indexed by string

initial begin
   string tom = "tom";
   age [tom] =  21;
   age ["joe"] =  32;
   $display("%s is %d years of age ", tom, age[tom], "[%0d ages available]", age.num());
end
```

What is the output ?

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Queues & Lists

- SV has a built-in list mechanism which is ideal for queues, stacks, etc.
- A list is basically a variable size array of **any** SV data type.

```
int q1[$];          // $ represents the 'upper' array boundary
int n, m, item;

                    ' (tick) signifies cast overload and is required

q1 = '{ n, q1 };    // uses concatenate syntax to write n to the left end of q1
q1 = '{ q1, m };    // uses concatenate syntax to write m to the right end of q1

item = q1[0];       // read leftmost ( first ) item "n" from list
item = q1[$];       // read rightmost ( last ) item "m" from list

n = q1.size();      // determine number of items on q1

q1 = q1[1:$];       //    ?
q1 = q1[0:$-1];     //    ?

for (int i=0; i < q1.size(); i++)// step through a list using integers (NO POINTERS)
  begin … end

q1 =  '{ };         // clear the q1 list
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Queue Methods

function int size()

- Returns the number of items in the queue. If the queue is empty, it returns 0.

function void insert (int index, queue_type item)

- Inserts the given item at the specified index position
- `Q.insert (i, e)  =>  Q = '{Q[0:i-1], e, Q[i:$]}`

function void delete (int index)

- Deletes the item at the specified index position
- `Q.delete (i)  =>  Q = '{Q[0:i-1], Q[i+1:$]}`

function queue_type pop_front()

- Removes and returns the first element of the queue
- `e = Q.pop_front ()  =>  e = Q[0]; Q = Q[1:$]`

function queue_type pop_back()

- Removes and returns the last element of the queue
- `e = Q.pop_back ()  =>  e = Q[$]; Q = Q[0:$-1]`

function void push_front (queue_type item);

- Inserts the given element at the front of the queue
- `Q.push_front (e)  =>  Q = '{e, Q}`

function void push_back (queue_type item);

- Inserts the given element at the end of the queue
- `Q.push_back (e)  =>  Q = '{Q, e}`

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Queue Example: Simple FIFO

```
module q_fifo( input  logic [7:0] data_in, output bit empty, full,
               logic [7:0] data_out,  input event write_fifo, read_fifo );

logic [7:0] q [$];                      // Declare the queue q
always @ (write_fifo)                   // the write_fifo event
  if(full)
    $display ("tried to write a full fifo");
  else begin
    q.push_front(data_in);              // write to left end of q
    if(q.size > 7)
      full = 1;                         // set as full
    empty = 0;                          // set as not empty
  end
always @ (read_fifo)                    // the read_fifo event
 if(empty)
   $display("tried to read an empty fifo");
 else begin
   data_out <= q.pop_back();            // remove from right end
   full = 0;                            // not full after a read
   if(q.size() == 0)
     empty = 1;                         // set as empty
 end
endmodule
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Structures

- Think of a structure as an object containing data members of any SV type

```
struct{
  bit[7:0]   my_byte;
  int  my_data;
  real pi;
} my_struct;    // my_byte, my_data and pi are "members" of my_struct
```

- *Data members can be referenced individually (using the . operator) or altogether as a unit*

```
initial begin
    my_struct.my_byte = 8'hab;
    my_struct = '{0, 99, 3.14};
end
```

Structures may:
- be packed or unpacked.
- be assigned as a whole
- pass to/from a function or task as a whole
- contain arrays

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Unpacked vs Packed Structures

Unpacked




```
struct {bit[1:0]a0;
        bit[2:0] a1;
        bit[5:0] a2;
        bit[8:0]a3;
       } u_pkt;
```

- Default for structs
- Tool dependant implementation.
- Think of as a logical grouping of member elements
- Each member may be assigned differently

Packed




```
struct packed  {
        bit[1:0] a0;
        bit[2:0] a1;
        bit[5:0] a2;
        bit[8:0] a3;
        } p_pkt;
```

- Much more useful in hardware
- Easily converted to bit-vectors
- May be accessed as a whole
- First member specified is most significant
- May be declared as signed for arithmetic

Limitations

- Unpacked arrays/structures are NOT allowed here

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

Multidimensional Arrays

- SystemVerilog supports multi-dimensional arrays just like Verilog…

```
bit [7:0] mem [4:1];     // byte-wide memory with 4 addresses, like Verilog
mem[ i ] [6:1] = 0;      // 2D+ indexing supported (like Verilog 2001)
```

- Also from Verilog 2001 we get multi-multi-dimensions… phew!

```
bit [a:b] [n:m] [p:q] mem [ t:u] [v:w] [x:y]; // arbitrary dimensions
```

packed unpacked

- The terms packed and unpacked to refer to how the data is actually stored in memory
  - packed => 8-bits to a byte, unpacked => 1 bit per word

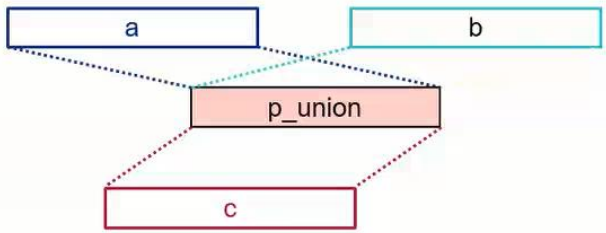


Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential
SIEMENS

Packed Unions

- A packed union contains 1 or more packed members
  - All members are the same size and occupy the same space
  - Data may be written via one member, read by another
  - Unlike C/C++ SV unions are independent of platform byte ordering
  - The union may be accessed as a whole

```
union packed  {
int     a;
integerb;
reg[31:0]  c;
} p_union;
```



- Characteristics
  - Easy conversion to bit-vectors
  - May be accessed as a whole
  - First member specified is most significant
  - May be declared as signed for arithmetic
  - Non-integer datatypes (e.g. real) are NOT allowed
  - Unpacked arrays/structures are NOT allowed
  - ***if any member is 4-state, all members are cast to 4-state***

40:47

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Packed Union - example

- Since all members are same size and occupy the same memory space
  - The same data may be accessed via any of the members
  - Permits one data element to exist in multiple name-spaces

```
typedef struct packed {          // Default unsigned
          bit [7:0] f1, f2;
          bit [11:0] f3;
          bit [2:0] f4;
          bit f5;
          bit [63:0] f6;
          } my_packed_struct;

typedef union packed {           // Default unsigned
          my_packed_struct mps;
          bit [95:0] bits;
          bit [11:0] [7:0] bytes;
          } my_packed_union;

my_packed_union u1;
```

3 members

Given:

```
byte b;

b = u1.bits[87:80];
```

equivalent!



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

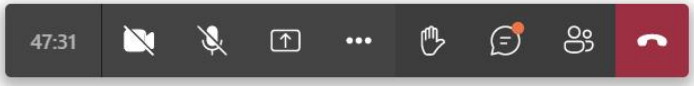
SIEMENS

SV Scheduler

In this section



Processes



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Abbas, Amr Mohamed (DI ...

Processes

- An SV description consists of connected threads of execution called processes
- Processes are objects
  - Can be evaluated
  - Can have state
  - Respond to changes on inputs
  - Produce outputs
- All processes are concurrent
  - Order of execution is indeterminate
- SV processes:
  - Procedural blocks
    - `initial, always, always_ff, always_comb, always_latch`
  - Primitives, continuous assignments, asynchronous tasks

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

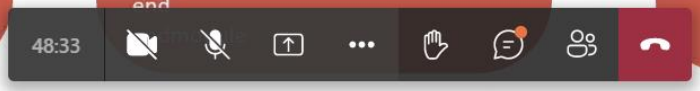
Specialized 'always' blocks

- The general purpose 'always' block is not intuitive enough to understand the type of logic implemented by it.
- To remove these problems, three different types of 'always' blocks are introduced in SystemVerilog, i.e. 'always_ff', 'always_comb' and 'always_latch', which are discussed below,

```
module always_comb_ex
( input logic a, b, output logic y, z );
always_comb
 begin
  if (a > b)
  begin
   y = 1; z = 0;
  end
  else if (a < b)
  begin
   y = 0; z = 1;
  end
  else
  begin
   y = 1; z = 1;
  end
end
endmodule
```

```
module always_latch_ex
( input logic a, b, output logic y, z );
always_latch
 begin
  if (a > b)
  begin
   y = 1; z = 0;
  end
  else if (a < b)
  begin
   y = 0;  // z = 1;
  end
  //else
  //begin
  //  y = 1; z = 1;
  //end
end
```

```
module always_ff_ex
( input logic clk, reset, output logic y,
z );
always_ff @(posedge clk, posedge
reset)
begin if (reset)
   begin
      y <= 0; z <= 0;
   end else
   begin
      y <= 1; z <= 1;
   end
end
endmodule
```



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

Program Control

In this section



Loops
disable / break / continue



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

do loop

Syntax:
```
do <statement> while <expression>
```

- Like a while loop, but evaluates after the loop executes rather than before

```
while (i < 10)
   begin
      ...
      i = i +1;
   end
```

Loop would NEVER execute if i >= 10 So programmer has to setup value of i before loop

```
do
   begin
      ...
      i = i +1;
   end
while (i < 10) ;
```

Guaranteed to execute AT LEAST once



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS

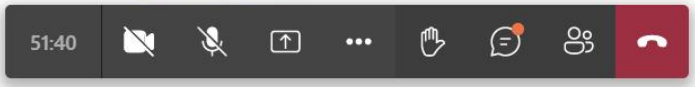
for - Loop Variable

- Verilog 2001:

```
int j;
initial
   for ( j = 0; j <= 20; j = j+1)
      // j is global and could be 'accidentally'
      // modified elsewhere
     $display(j);
```

- SystemVerilog allows the loop variable to be declared within the loop:

```
initial
   for ( int j = 0; j <= 20; j++)    // j is local
      $display(j);
```



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS



for – multi-assignments

- SystemVerilog allows multiple initialization or update assignments within the loop.
- Uses a simple comma-separated syntax.
- NOTE: All or none of the variables must be local

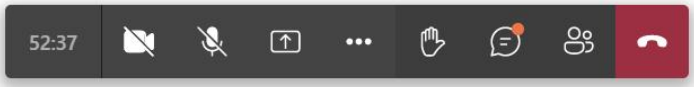
```
int k = 99;          // This declaration will be ignored by the following loop

initial
   for ( int j = 0, k = 10; j <= k; j++, k-- ) // j and k are local
       #10 $display("  j: %0d  k: %0d",j,k);

initial
  #40 $display("    Global k: %0d", k);
```

Simulator output

```
#  j: 0  k: 10
#  j: 1  k: 9
#  j: 2  k: 8
#    Global k: 99
#  j: 3  k: 7
#  j: 4  k: 6
#  j: 5  k: 5
```



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST I)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Abbas, Amr Mohamed (DI ...

foreach loop

Syntax:
```
foreach ( <array name>[<loop variables>] ) <statement>
```

- Iterates over all elements of an array
  - Array can be fixed-size, dynamic, or associative

```
int Ary[10];
foreach (Ary[i])
  $display("Ary[%d] = %d",i,Ary[i]);
```

i iterates from 0 to 9



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS



foreach – examples

- Multiple loop variables correspond to nested loops
- If used with associative arrays, the type of the loop variable is auto-cast to the type of the array index

```
int mem[8][4];
foreach (mem[i,j])
  $display("mem[%d][%d] = %d",i,j,mem[i][j]);
```

j iterates first from 0 to 3, then i from 0 to 7 as if in a nested for loop

```
logic[7:0] binmem [4][2];
foreach (binmem[row,col,b])
  $display("Bit %d of binmem[%d][%d] is %z",b,row,col,binmem[row][col][b]);
```

b iterates first from 7 down to 0, then col from 0 to 1, then row from 0 to 3

```
int assoc[string];
foreach (assoc[s])
  $display("assoc[%s] = %d",s,assoc[s]);
```

s is cast as a string and iterates through all keys in the associative array



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS



break & continue

- Instead, System Verilog adds C-like constructs: **break** and **continue**
  - These offer similar functionality but do NOT require an explicit block/task name.

```
reg[3:0] b, a;
…
for ( int i=0; i<52; i++ )
  begin
    a = 1;
    #5 case(b)
         1:  continue;
         0:  break;

      endcase
    a = 0;
  end
…
```

jump to next iteration of loop

exit the loop (as in C)

NOTE
break and **continue** may only be used inside a loop



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

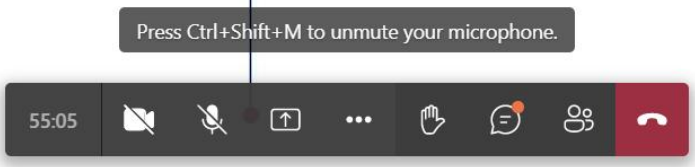
Abbas, Amr Mohamed (DI ...

Hierarchy

In this section



Ports
Driving SV variables Time specifiers Packages
Compilation Units



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Abbas, Amr Mohamed (DI ...

Implicit Port Connections

- SystemVerilog adds two new ways to reduce the chore of instantiation for the common situation where port and connected signal match in name and size
- Consider this 4:1 mux:

```
module mux (output logic [2:0] out,
            input logic [2:0] a, b, c, d, sel);
```

- Verilog 'by-name' instantiation (port d unconnected)

```
mux U1 (.out(out), .a(a), .b(b), .c(c), .d(), .sel(sel1));
```

- `.name` syntax instantiation

```
mux U2 ( .out, .a, .b, .c, .d(), .sel(sel1) );
```

- `.*` syntax instantiation

```
mux U3 ( .*
```

Unconnected port using 'by-name' syntax



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA Confidential

SIEMENS



Implicit Port Connections – 2

- Both new SystemVerilog styles of instantiation require:
  1. Ports and their connecting variables must have same name and same width
  2. Ports and their connecting variables must be of compatible type
  3. Ports outside of the implicit list must be connected by port name (not order)
- So, which to use, `.name` or `.*` ?
  - `.name` allows implicit connections but shows port & signal names for documentation
  - `.*` allows full wildcarding where listing port & signal names is not required



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Abbas, Amr Mohamed (DI ...

SV and hierarchy

- **Packages**
  - A mechanism for sharing parameters, data, type, task, function, sequence and property declarations amongst modules, interfaces and programs.
- **Compilation units**
  - A collection of one or more SystemVerilog source files compiled together
- **Compilation-unit scope**
  - A scope local to the compilation unit, containing all declarations that lie outside of any other SV scope
- **$unit::**
  - A name used to explicitly access the identifiers in the compilation-unit scope



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential
SIEMENS
Abbas, Amr Mohamed (DI ...

Packages -1

- Packages are explicitly named scopes declared at top of hierarchy
  - May contain types, variables, tasks, functions, sequences, and properties

```
package p;
   typedef enum { FALSE, TRUE } BOOL;
   const BOOL c = FALSE;
endpackage
```

```
package q;
   const int c = 0;
endpackage
```

Declaration assignments are performed ***before*** any always/initial/program blocks execute

- Package items may be referenced within modules, interfaces, etc. (even other packages)
  - Using fully resolved name (use the scope resolution operator "::")

```
module foo;
  wire a = q::c;
endmodule
```

:: is the scope resolution operator (q::c ➔ c inside of q)



Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS



Packages -2

- Packages are explicitly named scopes declared at top of hierarchy
  - May contain types, variables, tasks, functions, sequences, and properties

```
package p;
   typedef enum { FALSE, TRUE } BOOL;
   const BOOL c = FALSE;
endpackage
```

```
package q;
   const int c = 0;
endpackage
```

- Instead of a fully resolved reference an entire package or a package item may be imported
- However, if the identifier of an imported item is already used in the importing scope, that item is silently NOT imported

* is a wildcard, allowing ANY identifier in package to be referenced

```
module foo2;
  import q::*;
  wire a = c;       // no need to reference package q since c is "imported" already
  import p::c;  // does this error ?

  import p::*;      // c silently NOT imported, no error/warning posted

  wire b = p::c; // does this error ?
endmodule
```

01:00:04

Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Abbas, Amr Mohamed (DI ...

Package example

```
package pkg_rmac;

    typedef struct {bit[55:0] preamble ; bit [7:0] sfd ;} h_type;

    typedef enum {
            _,
            Preamble,
            SFD,
            Destination_MAC,
            Source_MAC,
            Length,
            Packet_Num,
            Payload,
            CRC
    } fids;

    // Typedefs for stream_mp3_file task
    typedef struct {
        int n64;
        int n244;
        int n428;
        int n608;
        int n792;
        int n972;
        int n1156;
        int n1340;
        int n1500;
    } distro;

    typedef struct {
        int sendmp3;
        int noise;
    } frm;

    typedef struct {
        int normal;
        int long;
        int short;
        int random;
    } gap;

    typedef struct {
        frm frame;
        distro dis;
        gap ifg;
    } stream_set;

endpackage
```

```
`include "tb_eth_defines.v"
`include "eth_defines.v"
`include "timescale.v"

module tb_rmac(
    input wire [3:0]MTxD,       // NOT USED
    . . .

    //================================
    //===  REFERENCE SV PACKAGE   ===
    //================================
    import pkg_rmac::*;

    stream_set set; // used to configure…

    . . .
```

Question

What is the advantage of using a pkg ins… of tb_eth_defines.v above?

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

SIEMENS

01:14:20

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)

Type here to search

25°C سماء صافية ENG 7:15 PM 3/5/2023

Compilation Unit (CU)

- A CU is a collection of SV source files intended to compile together
- The syntax to declare a CU is tool-specific
  - CU scope ($unit) is defined as local to the compilation unit, it contains all declarations that lie outside of any other scope

    Two use-models are supported:

    1) Each file is a separate CU (so declarations in each CU scope are accessible only within its corresponding file)
    2) All files on a given compilation command line make a single CU
       - Declarations within those files are accessible anywhere else within the implied CU)



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
25°C سماء صافية
ENG
AA
Abbas, Amr Mohamed (DI ...

CU Example -1

code.sv

```
typedef struct{ logic [9:0] addr;
                logic[31:0] data;
               } global_pkt;
global_pkt unit_pkt;

task global_task (input global_pkt in1);
   $display($stime,,"addr: %h, data: %h",in1.addr, in1.data);
endtask

module code;
logic [7:0] x;

global_pkt y;

initial
  begin
   #20;    y.data = 2;   y.addr = 2;   global_task(y);
   #10;
   unit_pkt.data = 3;
   $unit::unit_pkt.addr = 3;
   global_task(unit_pkt);

  end

endmodule
```

NOTICE **$unit::** is redundant

<os> vlog code.sv
<os> vsim –c code

```
# Loading work.code_sv_unit
# Loading work.code
VSIM 1> run -all
#   20000000 addr: 002, data: 00000002
#   30000000 addr: 003, data: 00000003

VSIM 2>
```

According to the default use model in SV each file is its own Compilation Unit…

so the typedef and task declarations are within the implicit CU of module code.



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA OST 1)
Type here to search
25°C سماء صافية
ENG
AA
Abbas, Amr Mohamed (DI ...

CU Example -2

code.sv

```
typedef struct{ logic [9:0] addr;
                logic[31:0] data;
              } global_pkt;
global_pkt unit_pkt;

task global_task (input global_pkt in1);
   $display($stime,,"addr: %h, data: %h",in1.addr, in1.data);
endtask

module code;
logic [7:0] x;

global_pkt y;

initial
  begin
  #20;
   y.data = 2;
   y.addr   =   2;
   global_task(y);

   #10;
   unit_pkt.data = 3;
   unit_pkt.addr = 3;
   global_task(unit_pkt);

  end

endmodule
```

<os> vlog code.sv test_1.sv.

test_1.sv

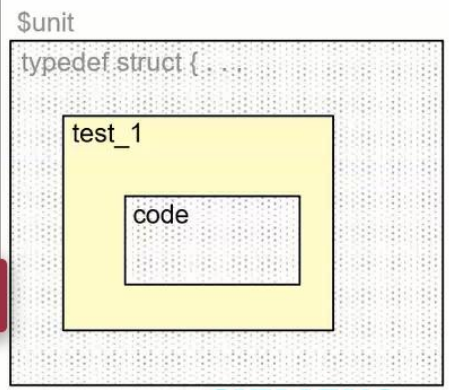
```
module test_1;

logic unit_pkt;  // local variable hides global

code U1();

initial
  begin
  #10;
   unit_pkt.data = 1; // not compat. with local variable type
   unit_pkt.addr = 1;
   global_task(unit_pkt);
  end
endmodule
```

NOTICE



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
23°C سماء صافية
ENG
mazen (Guest)

Dynamic Processes

In this section



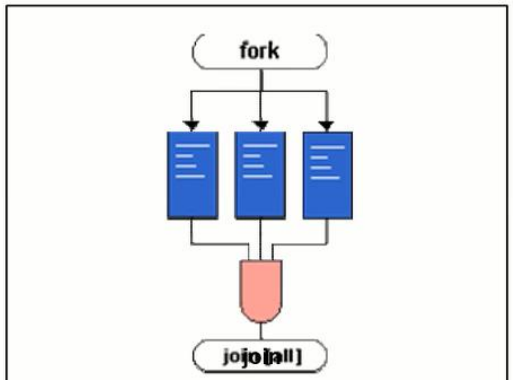
Dynamic processes
Process control

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

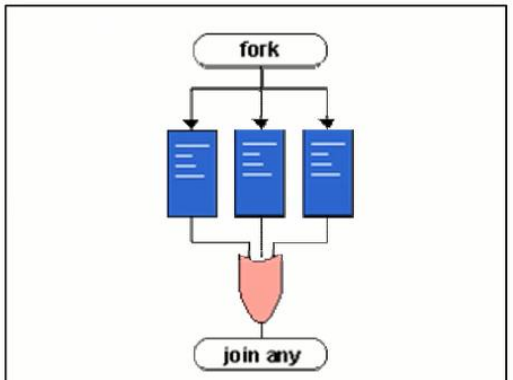
01:19:07
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
23°C سماء صافية
ENG
AA
Abbas, Amr Mohamed (DI ...

Dynamic Processes

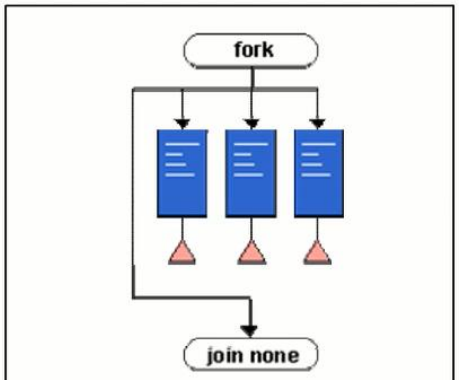
- SystemVerilog defines 2 new special cases of fork…join with associated keywords **`join_any`** & **`join_none`**




```
fork
  ...
  ...
  ...
join      // all blocks finished
```




```
fork
  ...
  ...
```




```
fork
  ...
  ...
  ...
join_none  // no waiting at all
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
01:19:21
23°C سماء صافية
ENG
Abbas, Amr Mohamed (DI ...

Process Control – `wait fork`

- With Dynamic processes SystemVerilog needed to provide more global detection that spawned processes have completed
- The **`wait fork`** statement is used to ensure that all child processes (spawned by the process where it is called) have completed execution

```
begin
  fork
    task1();
    task2();
  join_any       // continue when either task completes

  fork
    task3();
    task4();
  join_none      // continue regardless

  wait fork;     // block until tasks 1-4 complete
end
```



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA OST 1)
Type here to search
23°C سماء صافية
ENG
AA
Abbas, Amr Mohamed (DI ...

Process Control – `disable fork`

- The **`disable fork`** statement terminates ***all*** active child processes of the process where it is called,
  - Termination is recursive, in other words it terminates child processes, grandchild processes, etc.

```
task test_with_timeout;
  fork   // spawn off two child tasks in parallel, 1st to finish triggers off the join_any
    run_test();
    timeout( 1000 );
  join_any
  disable fork;   // kills the slower task
endtask
```

```
task test_with_timeout;   // Verilog 1995
  fork
    begin
      run_test();
      disable timeout;
    end
    begin
      timeout( 1000 );
      disable run_test;
    end
  join
endtask
```


Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1)
Type here to search
01:21:05
23°C سماء صافية
ENG
AA
Abbas, Amr Mohamed (DI ...

Together mode scene changed

Dismiss

Classes

In this section



- OOP concepts
- Inheritance
- Virtual Methods
- Reference
  - Encapsulation
  - Parameterization
  - Polymorphism

SIEMENS

Type here to search

23°C سماء صافية ENG 7:24 PM 3/5/2023

+75 K1 ES YH A1 Li M AZ AA HS

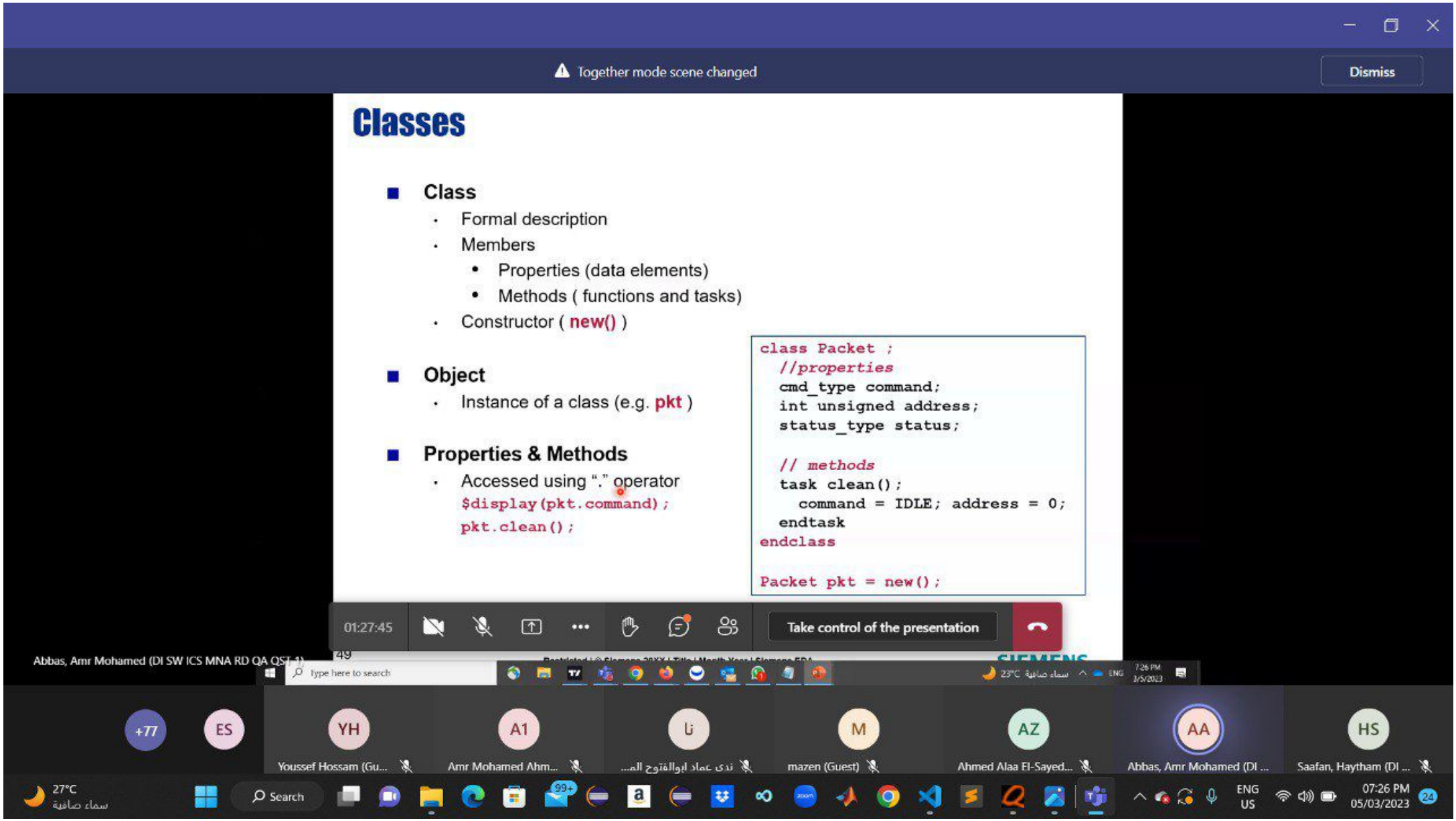
27°C سماء صافية Search ENG US 07:24 PM 05/03/2023

Together mode scene changed
Dismiss

SV Classes - Overview

- Object Oriented design is a common programming paradigm
  - Data and the means to manipulate is described together in a formal structure called a class
- A `class` is a datatype, similar to a `struct`
  - Has data elements (called `properties`)
  - But also contains functions and tasks (called `methods`) through which class properties may be manipulated
- An instance of a `class` is referred to as an `object`
- SystemVerilog objects are dynamically created and destroyed
  - Memory allocation and deallocation (garbage collection) is handled automatically
  - Since pointers are a key ingredient in the flexibility of classes, SystemVerilog implements them too, but in a safer form, called `handles`
- Code minimization and reuse is facilitated through inheritance, parameterization and polymorphism

Type here to search
+75 K1 ES YH A1 Li M AZ AA HS
27°C سماء صافية
Search
ENG US
07:25 PM 05/03/2023


Together mode scene changed
Dismiss
Classes
Class
Formal description
Members
Properties (data elements)
Methods (functions and tasks)
Constructor (new())
Object
Instance of a class (e.g. pkt)
Properties & Methods
Accessed using "." operator
$display(pkt.command);
pkt.clean();
class Packet ;
//properties
cmd_type command;
int unsigned address;
status_type status;
// methods
task clean();
command = IDLE; address = 0;
endtask
endclass
Packet pkt = new();
01:27:45
Take control of the presentation
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 4)
49
Youssef Hossam (Gu...
Amr Mohamed Ahm...
mazen (Guest)
Ahmed Alaa El-Sayed...
Abbas, Amr Mohamed (DI ...
Saafan, Haytham (DI ...
27°C
Search
ENG
US
07:26 PM
05/03/2023

Together mode scene changed
Dismiss

Class Constructors - `new()`

- Special member function called when an instance of a class is created
  - Name of function is `new()` (reserved keyword)
    - No return type specified in declaration
    - May have arguments - Allows for run-time customization
  - If no constructor is specified, compiler will create one
  - Only one constructor for each class
- Used for initialization of the instance

user-provided Constructor

Constructor args (optional), allow for run-time customization

```
class Packet ;
  //properties
  cmd_type command;
  int unsigned address;
  status_type status;

  //initialization
  function new(int addr);
    command = READ;
    address = addr;
    status = OK;
  endfunction
endclass
```


SIEMENS

+75 K1 ES YH A1 Li M AZ AA HS
27°C سماء صافية Search ENG US 07:28 PM 05/03/2023

Together mode scene changed
Dismiss

Class Object Creation (Instantiation) - 1

- To create a class object
  - **First** declare an object `handle` of the class type
    - Like a pointer

```
module test;
  Packet pkt;   // declare object handle, does not create object itself, its value is null
initial
  ...
endmodule
```



Type here to search
+75 K1 ES YH A1 Li M AZ AA HS
27°C سماء صافية
Search
ENG US
07:29 PM 05/03/2023

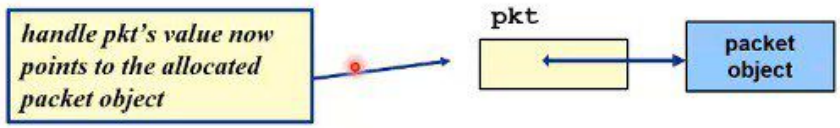
Together mode scene changed

Dismiss

Class Object Creation (Instantiation) - 2

- **Second** call the function **new()** and assign the return to the object handle
  - Dynamically allocates (creates) the object by calling the constructor for the class
  - Type of left hand side determines return type of **new()**

```
module test;
  Packet pkt = new(0);          // declare object handle and initialize it
 ...
endmodule
```




```
module test;
  Packet pkt;          // declare object handle, does not create object itself, just the handle

initial
  pkt = new(0);        // create object and assign to the handle pkt (pkt "points" to object created)
endmodule
```


SIEMENS

Type here to search

23°C سماء صافية ENG 7:30 PM 3/5/2023

+73 K1 ES YH A1 Li M AZ AA HS

27°C سماء صافية Search ENG US 07:30 PM 05/03/2023

Together mode scene changed
Dismiss

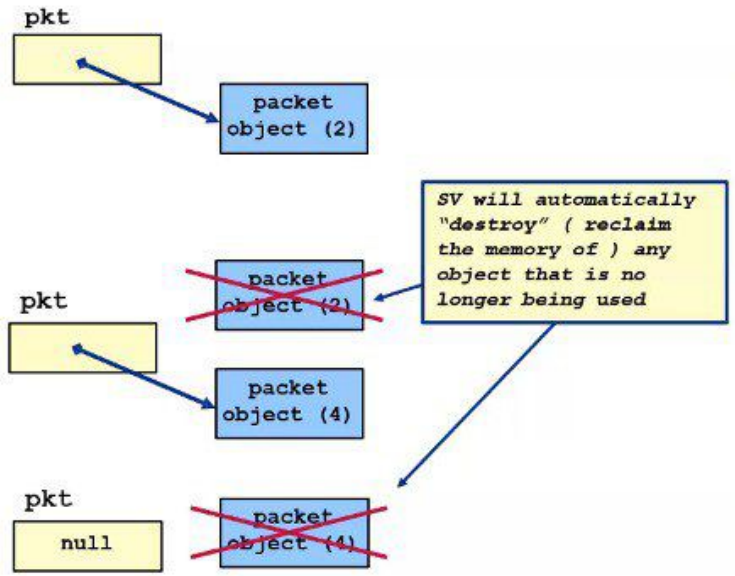
Re-Assigning Handles to New Objects

```
module test;
  Packet pkt;

initial begin
  pkt = new(2);


  ...
  pkt = new(4);


  ...
  pkt = null;
  end
endmodule
```



+73 K1 ES YH A1 Li M AZ AA HS
27°C سماء صافية
Search
ENG US
07:30 PM 05/03/2023

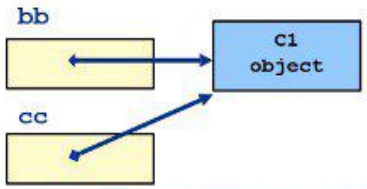
Together mode scene changed
Dismiss

Copying Objects

- OK, we can create/destroy objects… How about duplicating one?

```
class C1 ;
  Packet pp;
endclass

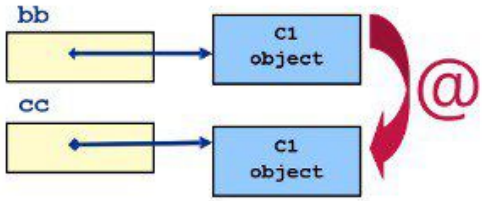
C1 bb;
bb = new();
C1 cc;
cc = bb;
```



@

Two handles pointing to the same object

```
C1 bb, cc;
bb = new();
cc = new bb;
```



@

Shallow Copy: Any nested objects will NOT be duplicated , only the members themselves (handle pp will be copied but not the object it points to)

01:34:23
Take control of the presentation
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 4)
54
SIEMENS
Type here to search
+74 ES AZ HS A1 M YH AA RK
Saafan, Haytham (DI ... Amr Mohamed Ahm... mazen (Guest) Youssef Hossam (Gu... Abbas, Amr Mohamed (DI ... Rufaidah Muhamma...
27°C سماء صافية
Search
ENG US
07:33 PM 05/03/2023

Together mode scene changed
Dismiss

this

- A predefined object handle that refers to the current object

```
class Packet;
 integer status;

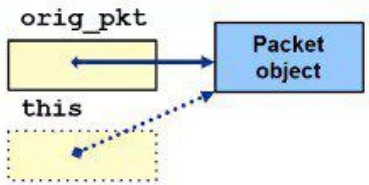
 virtual function Packet clone();

   Packet temp = new this;          // create new Packet object
   return(temp);                    // return cloned object
 endfunction

endclass

Packet orig_pkt, cloned_pkt;

initial begin
  Packet orig_pkt = new();
  orig_pkt.status = 55;
  cloned_pkt = orig_pkt.clone();
  $display("cloned_pkt.status = %0d", cloned_pkt.status);
end
```



Simulation output

```
# cloned_pkt.status = 55
```


SIEMENS

+74 ES AZ HS A1 Li M YH AA RK
Rufaidah Muham...
25°C سماء صافية
Search
ENG US
07:35 PM 05/03/2023

Class example

Press Esc to exit full screen

```
class ether_packet;

// Ethernet Packet Fields
  bit[55:0] preamble = 'h55555555555555;
  bit [7:0] sfd = 'hab;
  bit[47:0] dest, src;
  bit[15:0] len;
  bit [7:0] payld [ ];

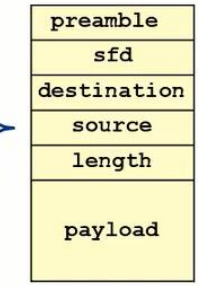
  function new(int i);
    payld = new[i];  len = i;
  endfunction : new

  task load_frame( input [47:0] lsrc, ldest,
                   input  [7:0] start_dat);
    src = lsrc;  dest = ldest;
    len = payld.size();
    if(start_dat > 0)
       for(int i = 0; i < len; i++)
           payld[i] = start_dat +i;
  endtask : load_frame

  function void print;
    $displayh("\t      src: ", src);
    $displayh("\t     dest: ", dest);
    $displayh("\t      len: ", payld.size);
    for(int i = 0; i < len; i++)
       $displayh("\t payld[%0h]: %0h",i,payld[i]);
    $displayh("");
  endfunction : print

endclass : ether_packet
```

Ethernet Packet




```
module test_ether();

ether_packet ep ;

initial
  begin
    ep = new (4);
    ep.load_frame('h55,'h66,'h77);
    ep.print();

//    . . .

    ep = new(1);
    ep.load_frame('h22,'h33,'h44);
    ep.print();

//    . . .

    ep = null;
  end

endmodule
```

Simulation output

```
     src: 000000000055
    dest: 000000000066
     len: 00000004
payld[0]: 77
payld[1]: 78
payld[2]: 79
payld[3]: 7a


     src: 000000000022
    dest: 000000000033
     len: 00000001
payld[0]: 44
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

SIEMENS

Type here to search
23°C سماء صافية
ENG
7:43 PM
3/5/2023

Interfaces

In this section



- Interface concepts & characteristics
- Simple bundled
- Methods
- Modports
- Importing methods



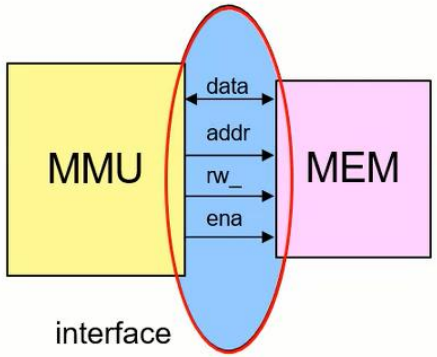
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

SIEMENS

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
23°C سماء صافية
ENG
7:46 PM

Interfaces

- Great simulation efficiency can be achieved by modeling the blocks of a system at different levels of abstraction, behavioral, rtl, gate, etc.
- In Verilog, the I/O between blocks has always remained at the lowest “pin” level
- High-performance system-level simulation requires abstract inter-block communication.



At its simplest an interface is like a module for ports/wires

```
module mmu(d, a, rw_, en);
  output [15:0] a;
  output rw_, en;
  inout [7:0] d;
  . . .
endmodule

module mem(d, a, rw_, en);
  input [15:0] a;
  input rw_, en;
  inout [7:0] d;
  . . .
endmodule

module system;
  tri [7:0] data;
  wire [15:0] addr;
  wire ena, rw_;

  mmu U1 (data, addr, rw_, ena);
  mem U2 (data, addr, rw_, ena);
endmodule
```

Traditional Verilog

```
interface interf;
  tri [7:0] data;
  logic [15:0] addr;
  logic ena, rw_;
endinterface

module mmu(interf io);
  io.addr <= ad;
  . . .
endmodule

module mem(interf io);
   adr = io.addr;
 . . .
endmodule

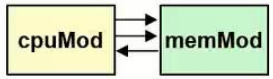
module system;
  interf i1;
  mmu U1 (i1);
  mem U2 (i1);
endmodule
```

SystemVerilog

SIEMENS

Type here to search
23°C سماء صافية
ENG
7:47 PM
3/5/2023

Design Example – Traditional Verilog




```
module memMod (input bit req,   bit clk,   bit start,
                     logic[1:0] mode,   logic[7:0] addr,
                    inout logic[7:0] data,
               output bit gnt,   bit rdy          );
   logic avail;
   ...
endmodule
```

module definition

Complete portlist…

```
module   cpuMod ( input bit clk,   bit gnt,   bit rdy,
                  inout logic [7:0] data,
                 output bit req, bit start,
                      logic[7:0] addr,
                      logic[1:0] mode   );
   ...
endmodule
```

module definition

Complete portlist…

```
module top;
   logic req, gnt, start, rdy; // req is logic not bit here
   logic clk = 0;
   logic [1:0] mode;
   logic [7:0] addr, data;

   memMod mem(req, clk, start, mode, addr, data, gnt, rdy);
   cpuMod cpu(clk, gnt, rdy, data, req, start, addr, mode);

endmodule
```

Top-level module definition

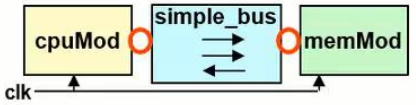
Signals to interconnect instances

Instantiate/connect everything

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

01:48:59
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
Abbas, Amr Mohamed (DI ...

Simple Bundle interface




```
interface simple_bus; // Define the interface
  logic req, gnt;
  logic [7:0] addr, data;
  logic [1:0] mode;
  logic start, rdy;
endinterface: simple_bus
```

When an interface is used like a port… all the signals of that interface are assumed to be **inout** ports

simple_bus Interface definition

signals making up simple_bus

```
module memMod (simple_bus a,     // port a is of 'type' simple_bus
               input bit clk); // and separately hook up clk
  logic avail;
 // a.req is the req signal in the 'simple_bus' interface

  always @(posedge clk) a.gnt <= a.req & avail;

endmodule
```

Declare a module and name its simple_bus interface 'a'

Access interface signals by name.signal

```
module cpuMod( simple_bus b, input bit clk);
  ...
endmodule
```

Declare a module and name its simple_bus interface 'b'

Top-level module

Instantiate/connect everything

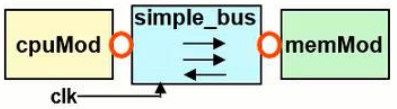
```
module top;
  logic clk = 0;

  simple_bus sb_intf;          // Instantiate the interface
  memMod mem (sb_intf, clk);   // Connect the interface to the module instance
  cpuMod cpu (.b( sb_intf)     clk(clk)   // Either by position or by n
endmodule
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

01:49:59
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافي غالبًا
ENG
Abbas, Amr Mohamed (DI ...

Interface Ports



- Ports may be defined to an interface
  - This allows external connections to the interface to be made and hence automatically to all modules which bind to the interface
    - Typically used for clk, reset, etc.

```
interface simple_bus (input bit clk); // Define the interface
   logic req, gnt, start, rdy;
   logic [7:0] addr, data;
   logic [1:0] mode;
endinterface: simple_bus
```

simple_bus Interface with one input port 'clk'

```
module memMod( simple_bus a );     // Uses just the interface
   logic avail;
   always @(posedge a.clk)         // the clk signal from the interface
      a.gnt <= a.req & avail;      // a.req is in the 'simple_bus' interface
endmodule
```

Module with interface 'a'

```
module cpuMod(simple_bus b);   ...   endmodule
```

```
module top;
   logic clk = 0;
   simple_bus sb_intf1(clk);       // Instantiate the interface
   simple_bus sb_intf2(clk);       // Instantiate the interface
   memMod mem1(.a(sb_intf1));      // Connect bus 1 to memory 1
   cpuMod cpu1(.b(sb_intf1));
   memMod mem2(.a(sb_intf2));
   cpuMod cpu2(.b(sb_intf2));
endmodule
```

2 simple_bus instances

cpu/memory pair 1

cpu/memory pair 2

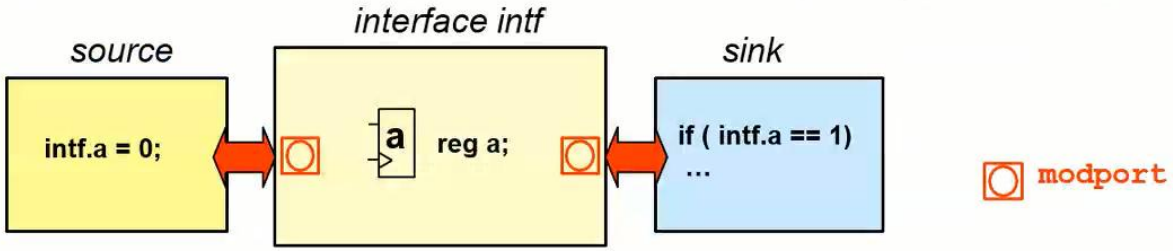
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

01:51:06
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافي غالبًا
ENG
AA
Abbas, Amr Mohamed (DI ...

Interface Modports

Modport derives from keywords “module” and “port”

They identify signal ownership from point of view of a module using that modport





- Modports specifically control variable access
- source/sink connect to a specific modport
- source/sink can’t use resources not listed in their modport



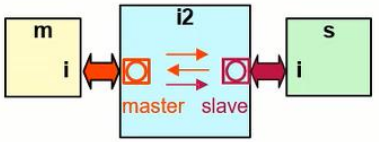
- Multiple modules can connect to interface using the same mo… … in our implementation

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

01:56:36
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
Abbas, Amr Mohamed (DI …

Modports – Example

"modport" keyword implies how the ports are accessed from the point of view of the module…




```
interface i2;
   logic a, b, c, d, e, f;
   modport master (input a, b, output c, d, e);
   modport slave (output a, b, input c, d, f);
endinterface
```

Interface 'i2' with 'master' and 'slave' modports

The modport list name can be specified in a couple ways:

Style 1 ***… in the module header…***

```
module m (i2.master i);
   ...
endmodule

module s (i2.slave i);
   ...
endmodule

module top;
   i2 itf;
   m u1(.i(itf));
   s u2(.i(itf));
endmodule
```

i is port name of interface i2

Modport indicates direction

Interface name acts as a type

Style 2 ***… in the module instantiation…***

```
module m (i2 i);
   ...
endmodule

module s (i2 i);
   ...
endmodule

module top;
   i2 itf;
   m u1(.i(itf.master));
   s u2( .i(itf.slave));
endmodule
```

01:57:17

HINT: Any number of modules may connect to an interface via the same modport

Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1)
Type here to search
22°C صافي غالبًا
ENG

Of what use are Interfaces in Verification ?

- **Interface**
  - Bundle signals into appropriate groups
  - Efficient way for objects to talk to modules/ports
  - Used to connect between class objects and modules (via virtual interface)
- **Modport**
  - Subgroup signals by purpose, timing and direction
  - Specify direction of asynchronous signals
  - Note: signals may appear in multiple modports



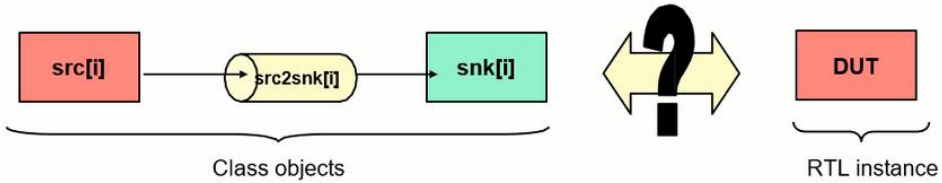
Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)

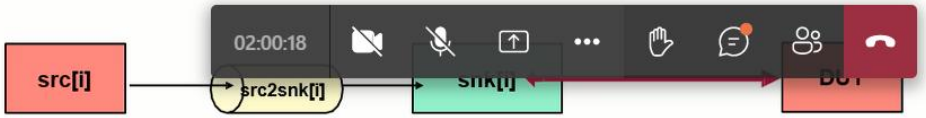
Type here to search
22°C صافٍ غالبًا
ENG

Objects Communicating with Module Instances

- How do you "talk" between a class object and a module?
  - Module instance has RTL ports
  - Class object does not have ports



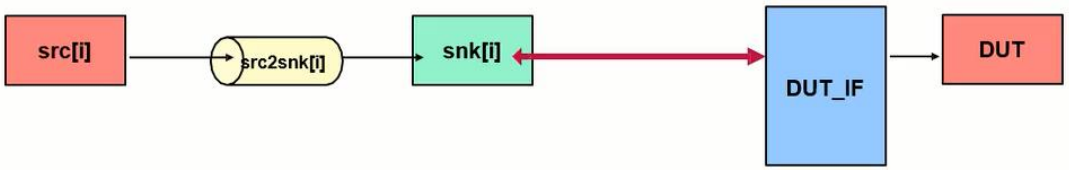
- Simple solution is to hierarchically address the module's ports (or signals connected to the ports) from the class
  - Issues
    - Must hard code into class definition a hierarchical path to the port or signal
    - Limits reusability of class
      - What if you have multiple DUT / snk's to connect?



Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافي غالبًا
ENG
AA

Classes & Interfaces

- An Interface can simplify the class object → module instance communication
  - Must still hard code into the class the hierarchical path to the interface instance

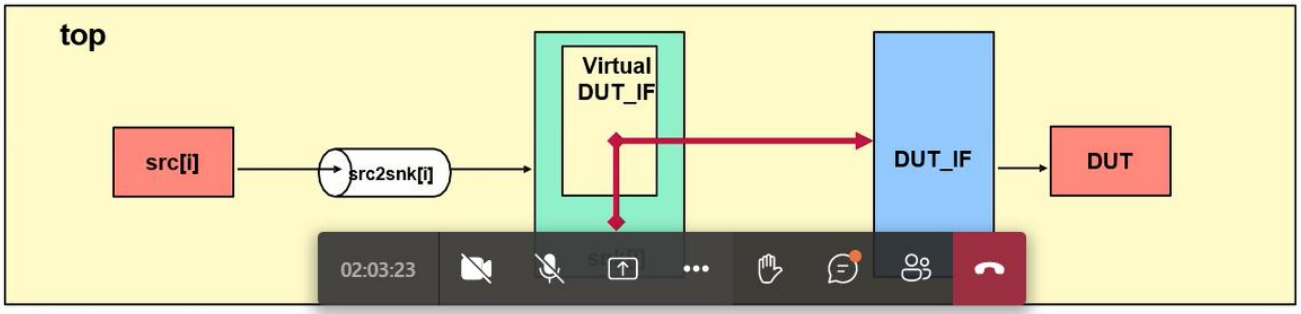


- Need a better approach….
  - If only interfaces were object oriented…

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Classes & Virtual Interfaces

- **Virtual Interface**
  - Mechanism for separating abstract models and test programs from the actual signals that make up the design
  - Can be declared as a class property
    - **Think of it as a handle to an interface**
    - May be initialized
      - Procedurally
      - By constructor argument
    - May be changed dynamically
    - Instead of referring directly (via a hierarchical pathname) to the actual signals in an interface the user manipulates virtual signals through the variable



Declaration syntax: `virtual interface_type v_if_name ;`

Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1)
Type here to search
22°C صافي غالبًا
ENG
AA
Abbas, Amr Mohamed (DI ...

Virtual Interface Example – Source/Sink

```
interface dut_if;                                    (1)
int data_rcvd;
  bit clk = 1;

  always #25 clk = !clk;

endinterface
```

```
module dut( input int data_rcvd,
            input bit clk );
always @(posedge clk)
begin
  $display("%m received data:   %0d",
  data_rcvd);
end
endmodule
```

```
module top;
import types::*;

dut_if d_if();                                       (2)

dut DUT(.data_rcvd (d_if.data_rcvd),
        .clk ( d_if.clk )                            (3)
);

sink        snk = new(d_if);                         (7)

source     src = new(2);
mailbox #(Packet) src2snk = new(1);

endmodule
```

```
class sink;
  mailbox #(Packet) in_chan; // null handle
  virtual dut_if  v_dut_if;                          (4)
  int id = 1;

  function new ( virtual dut_if   real_dut_if );     (5)
     // map virtual interface to real interface
     v_dut_if = real_dut_if;                         (6)
  endfunction

  task run();
    while(1) begin
      in_chan.get(stim_pkt);
    // access real interface via virtual IF
    @( negedge v_dut_if.clk )
        v_dut_if.data_rcvd = stim_pkt.field1;
    $display ("sink: Received packet with field1 = (%0d)",
                                    id, stim_pkt.field1);
    end
  endtask
endclass : sink
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

02:05:16

Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)

Type here to search

22°C صافٍ غالبًا ENG

AA

Abbas, Amr Mohamed (DI ...

Classes (cont.)

In this section



- Parameterization
- Inheritance
- Virtual Methods
- Reference
  - Encapsulation
  - Parameterization
  - Polymorphism

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

SIEMENS

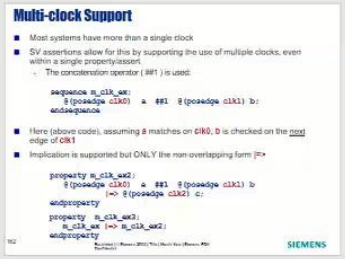
Type here to search
22°C صافٍ غالبًا
ENG
8:15 PM
3/5/2023



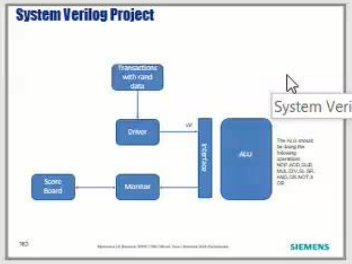
161



162



163



System Verilog Project

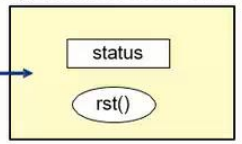
Inheritance

- Rather than add additional functionality to a class by copying and modifying we "extend" classes by inheritance

```
class Packet ;
  //properties
  integer status;

  // methods
  task rst();
    status = 0;
  endtask
endclass
```

Base class with properties/ methods

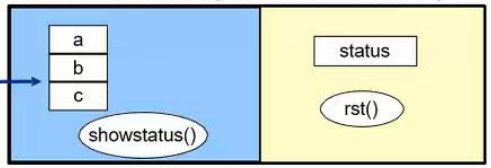


Inherited properties/ methods

```
class DerivedPacket extends Packet ;
  //properties
  integer a,b,c;

  // methods
  task showstatus();
    $display(status);
  endtask
endclass
```

Additional properties/ methods



- DerivedPacket inherits all of Packet, that is to say DerivedPacket "*is a* " Packet
- *PLUS* DerivedPacket has its own additional properties/methods

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
Confidential

SIEMENS

Click to add notes

02:24:50

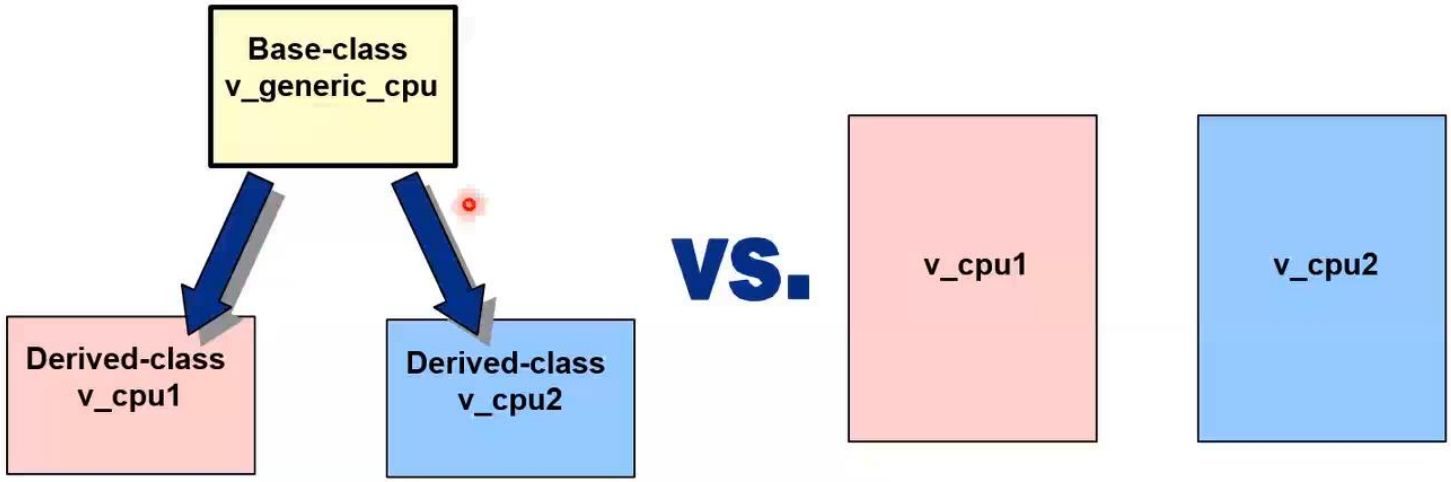
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)

Type here to search

22°C صافي غالبًا | ENG | 8:26 PM 3/5/2023

Inheritance Example

- Inheritance approach is ideal for verification
- Non-inheritance approach involves duplicated effort



- Base class with all common "stuff"
  - Customized v_cpu1
  - Customized v_cpu2
- Easy to make a class for verification of a 3rd type cpu
- Two different custom v_cpu's
  - Lots of duplicate effort
- Lots of work to create a third
- en more duplicate effort

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

02:32:55
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
Abbas, Amr Mohamed (DI ...

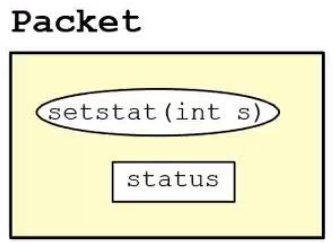
Inheritance – overriding

- Derived classes may “override” the definition of a member inherited from the base class
  - **"Hides"** the overridden property or method
  - To override a method the child method's signature must match the parent's exactly

```
class Packet ;
  int status = 4;

  function void setstat(int s);
    status=s;
    $display ("parent setstat");
  endfunction
endclass
```

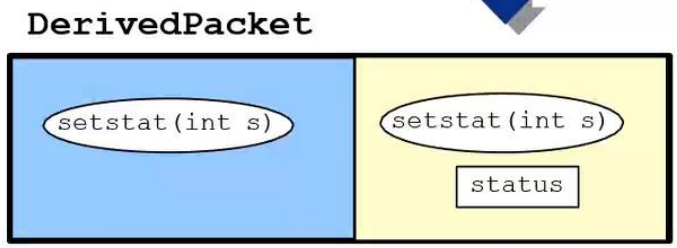
Base class with properties/ methods




```
class DerivedPacket extends Packet

  function void setstat(int s);
    status=s;
    $display ("child setstat");
  endfunction
endclass
```

DerivedPacket overrides method of Packet




```
DerivedPacket p = new();
initial begin
  p.setstat(33);   // output is "child setstat"
end
```

which setstat() is called?

K to access inherited property

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
SIEMENS
02:33:24
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
8:35 PM
3/5/2023

super

- The **super** keyword is used to refer to members of the base class
  - Required to access members of the parent class when the members are overridden in the child class
  - You may only "reach up" one level (super.super is not allowed)
  - Only legal from within the derived class

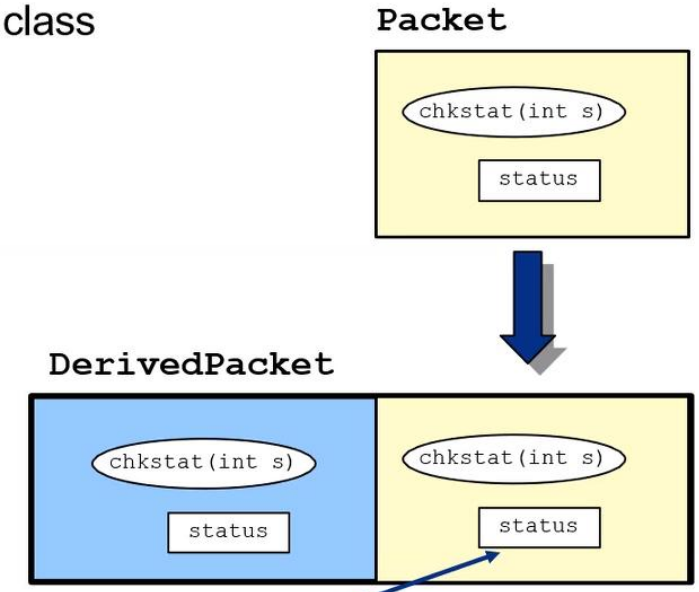
```
class Packet ;
  int status = 4;

  function bit chkstat(int s);
    return (status == s);
  endfunction
endclass
```

```
class DerivedPacket extends Packet ;
  int status = 15;

  function void chkstat(int s);
    $display(super.status);
  endfunction
endclass
```

Prints "4" because it explicitly refers to the base class member status



Cannot access status outside of object. super keyword legal only inside an object

```
DerivedPacket p = new();
p.super.status = 88;  //illegal!
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
SIEMENS

Type here to search
22°C صافي غالبًا
ENG
8:36 PM
3/5/2023

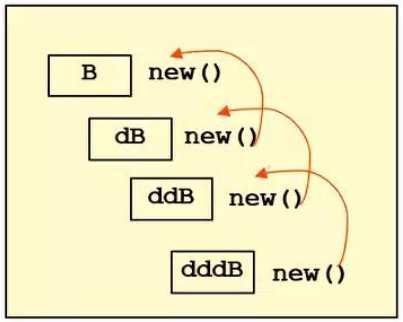
Constructors & Inheritance - 1

- All classes have a default constructor which may be overridden if specific initialization is required.

```
class Packet ;
  //properties
  integer status;

  // methods
  function new();
    status = 0;
  endfunction
endclass
```

Base class constructor




```
class DerivedPacket extends Packet ;
  //properties
  integer a,b,c;

  // methods
  function new();
    a = 0; b = 0; c = 0;
  endfunction

endclass
```

Derived class with its own constructor

When invoked, & before running any of its own code, new() invokes the new() method of its super-class, and so on up the hierarchy. Execution is root downward...

Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA
SIEMENS
Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1))
Type here to search
22°C صافٍ غالبًا
ENG
8:37 PM

Constructors & Inheritance - 2

- The default SV constructor has no arguments. What if a base class constructor requires arguments not present in the child class constructor?

```
class Packet;
  //properties
  integer status;
  int a,b,c;
  // methods
  function new(int i);
    status = 0;
    a = i; b = i; c = i;
 endfunction
endclass
```

Base class constructor

```
class DerivedPacket extends Packet;
  //properties
  // methods
  function new();
  endfunction

endclass
```

Does this pass compilation

```
class DerivedPacket1 extends Packet;
  //properties
  // methods
  function new();
    super.new(5);
  endfunction

endclass
```

```
class DerivedPacket2 extends Packet(5);
  //properties
  // methods
  function new();
  endfunction

endclass
```


Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

SIEMENS

Type here to search
22°C صافٍ غالبًا
ENG
8:39 PM
3/5/2023

Inheritance Puzzle #1

- Inheritance is quite straightforward but can raise some confusing situations…Consider:

```
class Packet ;

  task build_payld();
      $display("Packet payld");
  endtask

  task build_packet();
      …
      build_payld() ; // which build_payld() will this call?
      …
  endtask

endclass
```

```
class DerivedPacket extends Packet ;

   task build_payld() ; // over-ridden method
      $display("DerivedPacket payld");
   endtask

endclass
```

```
module poly1;

DerivedPacket der = new();

initial
  der.build_packet();

endmodule
```



Restricted | © Siemens 20XX | Title | Month Year | Siemens EDA

Parameterized Classes: Specialization

- A class which has parameters is called a ***generic class***
  - A generic class in and of itself is *not* considered a new type
  - An instance of a generic class with a unique set of parameters is called a ***specialization***
- A **specialization** *is* considered a new type
  - Each specialization with a different unique set of parameters defines a new type
  - A specialization of a generic class using the default parameter values/types is called a ***default specialization***
  - Two specializations of a particular generic class with *same* parameter values/types are considered the *same* type

```
AA aa_512 = new();                  // Create a default specialization of generic class AA
AA #(.size(256)) my_aa = new();     // Create a specialization of generic class AA (a new type)
initial begin
//  my_aa = aa_512;                 // Illegal - my_aa and aa_512 are different types
```

Questa runtime output from above code:

```
# ** Fatal: Illegal assignment to object of class AA__1 from object of class AA__2
```


© Willamette HDL Inc. 2007

02:44:02
Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
AA
Abbas, Amr Mohamed (DI ...

Parameterized Classes

- Constructors allow for customization of objects at run time
- Parameters allow for customization of objects at compile time
- Parameters can be value parameters or type paramters
- Parameterization of a value:

```
class AA #( int size = 512 );
  byte payload [size];
endclass
AA #(.size(256)) my_aa = new();      // my_aa has array size of 256
```

size substituted by the value 512 at compile time

- Parameterization of a type:

```
class BB #( type T = int );
  T payload [512];
endclass
BB bb_int = new();                   // object containing 512 ints
BB #(string) bb_string = new();      // object containing 512 strings
```

T substituted by the type int at compile time

- Multiple parameters:

```
class CC #( type T = int, int size = 512 );
  T payload [size];
endclass
CC #(.T(integer), .size(1024)) cc_handle = new();  // type integer, size 1024
```


02:45:12
© Willamette HDL Inc. 2007
SIEMENS
Abbas, Amr Mohamed (DI SW ICS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
8:46 PM
3/5/2023

Parameterized Classes: Inheritance

- Parameterized classes may be inherited

```
class D_base #( type T = int );
  T payload [512];
endclass
```

Generic class

T is int

```
//class D_1 derived from default specialization of D_base
class D_1 extends D_base ;   endclass
D_1 D_1_handle = new();
```

T is string

```
//class D_2 derived from specialization of D_base
class D_2 extends D_base #(.T(string) );   endclass
D_2 D_2_handle = new();
```

T is int
R is bit

```
//generic class D_3 derived from default specialization of D_base
class D_3 #( type R = bit ) extends D_base;  endclass
D_3  D_3_handle = new();
```

T is R
which is
byte

```
//generic class D_4 derived from specialization of D_base
class D_4 #( type R = bit ) extends D_base #(.T(R));
endclass
D_4 #(.R(byte)) D_4_handle = new();
```

T is R

© Willamette HDL Inc. 2007

02:47:26
Abbas, Amr Mohamed (DI SW CS MNA RD QA QST 1)
Type here to search
22°C صافٍ غالبًا
ENG
AA
Abbas, Amr Mohamed (DI ...

Polymorphism

- Consider a base class that is multiply-derived into sub-classes, each of which over-rides a common “virtual” base method
- Polymorphism allows that any derived-class object referenced via the base class will invoke the appropriate method without any special programmer intervention.

```
class Display;
integer v;

  virtual task Print();
     $display("v (dec): ", v);
  endtask

endclass
```

Base class is simply used for its handle… …never instantiated…

```
class HexDisplay extends Display ;

    task Print(); // over-ridden method
      $displayh("v (hex) : ",v);
    endtask

endclass
```

```
class OctDisplay extends Display ;

   task Print(); // over-ridden method
     $displayo("v (oct)
   endtask

endclass
```

```
module poly5;

HexDisplay hx = new();
OctDisplay  oc = new();
Display poly;

initial begin
  hx.v = 'habcd;
  oc.v = 'habcd;

  poly = hx;
  poly.Print();

  poly = oc;
  poly.Print();
end

endmodule
```

Simulation output

```
v (hex) : 0000abcd
v (oct) : 00000125715
```


© Willamette HDL, Inc. 2007

02:51:12
Abbas, Amr Mohamed (DI SW (CS MNA RD QA QST 1)
Type here to search
21°C صافي غالبًا
ENG
Abbas, Amr Mohamed (DI ...